

PAI | Sessional - I Exam Solution

Q1.

```
inventory = [  
  {  
    "title": "The Alchemist",  
    "author": "Paulo Coelho",  
    "price": 1200,  
    "stock": 15  
  },  
  {  
    "title": "Forty Rules of Love",  
    "author": "Elif Shafak",  
    "price": 1500,  
    "stock": 25  
  },  
  {  
    "title": "Sapiens",  
    "author": "Yuval Noah Harari",  
    "price": 2200,  
    "stock": 10  
  },  
  {  
    "title": "Atomic Habits",  
    "author": "James Clear",  
    "price": 1800,  
    "stock": 30  
  }  
]
```

```
print("=== Karachi Readers' Inventory Tool ===")

print("\nCHECK BOOK STOCK")
book_to_find = input("Enter the title of the book to check: ")

book_found = False

for book in inventory:

    if book["title"].lower() == book_to_find.lower():
        print(f"Stock for '{book['title']}': {book['stock']} copies available.")
        book_found = True
        break

if not book_found:
    print(f"'Book not found.'")

print("\nCALCULATE TOTAL INVENTORY VALUE")
total_value = 0

for book in inventory:
    book_value = book["price"] * book["stock"]
    total_value += book_value

print(f"-> The total value of all books is: PKR {total_value:,.2f}")

print("\nGENERATE REORDER LIST (15 or fewer copies)")
```

```
items_to_reorder = False

for book in inventory:
    if book["stock"] <= 15:
        print(f"-> {book['title']} (Stock: {book['stock']})")
        items_to_reorder = True

if not items_to_reorder:
    print("-> No books need to be reordered at this time.")
```

Output
=== Karachi Readers' Inventory Tool === CHECK BOOK STOCK Enter the title of the book to check: Sapiens Stock for 'Sapiens': 10 copies available. CALCULATE TOTAL INVENTORY VALUE -> The total value of all books is: PKR 131,500.00 GENERATE REORDER LIST (15 or fewer copies) -> The Alchemist (Stock: 15) -> Sapiens (Stock: 10)

Q2:

```
import math

warehouses = [
    [2, 8],
    [9, 1],
    [12, 11],
    [1, 2],
    [7, 6]
]

# 1. Create a Distance Function (using lists)
def calculateDistance(pointOne, pointTwo):
    x1 = pointOne[0]
    y1 = pointOne[1]
    x2 = pointTwo[0]
    y2 = pointTwo[1]
    distance = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
    return distance

print("=== Karachi Quick Ship's Warehouse Finder ===")
print("Enter your current X and Y coordinates or type 'quit' to exit.")

while True:
    xInput = input("\nEnter your X coordinate: ")

    if xInput.lower() == 'quit':
        print("Exiting tool. Safe travels!")
```

```
        break

yInput = input("Enter your Y coordinate: ")
if yInput.lower() == 'quit':
    print("Exiting tool. Safe travels!")
    break

driverX = float(xInput)
driverY = float(yInput)

# The driver's location is now stored in a list.
driverLocation = [driverX, driverY]

distances = [calculateDistance(driverLocation, wh) for wh in warehouses]

minDistance = min(distances)
minIndex = distances.index(minDistance)

nearestWarehouse = warehouses[minIndex]

print(f"The nearest warehouse is at {nearestWarehouse} with a distance of {minDistance:.2f} units.")
```

Output

```
=== Karachi Quick Ship's Warehouse Finder ===  
Enter your current X and Y coordinates or type 'quit' to exit.  
  
Enter your X coordinate: 15  
Enter your Y coordinate: 10  
The nearest warehouse is at [12, 11] with a distance of 3.16 units.  
  
Enter your X coordinate: 25  
Enter your Y coordinate: 40  
The nearest warehouse is at [12, 11] with a distance of 31.78 units.  
  
Enter your X coordinate: 5  
Enter your Y coordinate: 6  
The nearest warehouse is at [7, 6] with a distance of 2.00 units.  
  
Enter your X coordinate: 22  
Enter your Y coordinate: 40  
The nearest warehouse is at [12, 11] with a distance of 30.68 units.  
  
Enter your X coordinate: quit  
Exiting tool. Safe travels!
```

Q3:

```
class Faculty:
    def __init__(self, name, employeeId):
        self.name = name
        self.employeeId = employeeId

class Course:
    def __init__(self, courseCode, courseTitle):
        self.courseCode = courseCode
        self.courseTitle = courseTitle

class Department:
    def __init__(self, name):
        self.name = name
        self.courses = []
        self.facultyMembers = []

    def addCourse(self, courseCode, courseTitle):
        newCourse = Course(courseCode, courseTitle)
        self.courses.append(newCourse)

    def assignFaculty(self, faculty):
        self.facultyMembers.append(faculty)

    def displayDetails(self):
        print(f"Department: {self.name}")
        print("=== Courses Offered ===")
        for course in self.courses:
```

```

        print(f"    - {course.courseCode}: {course.courseTitle}")

print("== Assigned Faculty Members ==")
for prof in self.facultyMembers:
    print(f"    - {prof.name} (ID: {prof.employeeId})")

prof1 = Faculty("Dr. Muhammad Rafi", "EMP000")
lect1 = Faculty("Mr. Talha Shahid", "EMP001")

aiDepartment = Department("Computer Science")

aiDepartment.addCourse("DS-5007", "Natural Language Processing")
aiDepartment.addCourse("AI-2001", "Programming for AI")

aiDepartment.assignFaculty(prof1)
aiDepartment.assignFaculty(lect1)

aiDepartment.displayDetails()

```

Output
Department: Computer Science === Courses Offered === - DS-5007: Natural Language Processing - AI-2001: Programming for AI == Assigned Faculty Members == - Dr. Muhammad Rafi (ID: EMP000) - Mr. Talha Shahid (ID: EMP001)